Task 1 — User Registration & Login

Overview

Build a secure authentication slice that lets a new visitor create an account and lets an existing user sign in. This task exercises input validation, password hashing, token/session issuance, and negative-path handling across the full stack.

User Story

> ***As a*** *new visitor,* ***I want*** *to register an account and log in securely,* ***so that*** *I can access personalized features of the application.*

Your Story Point Estimation (To Be Completed by Learner)

Estimate the effort for this user story using a Fibonacci scale (1, 2, 3, 5, 8, 13, …). Consider complexity, effort, risk, and uncertainty, then justify your reasoning.

Factor	Your Assessment
Complexity *(how hard is the logic?)*	*(fill in)*
Effort *(how much work / how many parts?)*	*(fill in)*
Uncertainty / Risk *(unknowns, dependencies?)*	*(fill in)*
Your Estimate (points)	*(fill in)*
Rationale	*(explain how you arrived at your estimate)*

Acceptance Criteria

- Email must be unique and format-validated; password must be at least 8 characters and contain at least one letter and one number.
- Passwords are stored **hashed with a salt** (e.g., bcrypt/argon2/scrypt) — never in plaintext or reversible encryption.
- Successful registration creates the user and returns `201` with a safe user representation (no password/hash exposed).
- Successful login returns an auth token (JWT or opaque) or a session, plus an expiry.
- Wrong credentials return `401` with a **generic** message ("Invalid email or password") to avoid user enumeration.
- Duplicate registration is rejected with `409` and a clear message.
- Input errors return `400` with per-field validation messages.

Data Model

Field	Type	Constraints
`id`	UUID / int	Primary key
`email`	string	Unique, indexed, lowercased
`passwordHash`	string	Never returned in responses
`createdAt`	timestamp	Set on insert

API Contract (Framework Agnostic)

`POST /register`

- Request: `{ "email": "a@b.com", "password": "Secret123" }`
- Success `201`: `{ "id": "...", "email": "a@b.com", "createdAt": "..." }`
- Errors: `400` (validation), `409` (email exists)

`POST /login`

- Request: `{ "email": "a@b.com", "password": "Secret123" }`
- Success `200`: `{ "token": "...", "expiresIn": 3600, "user": { "id": "...", "email": "..." } }`

- Errors: `400` (validation), `401` (bad credentials)

Back-End Requirements

- Validate and normalize input (trim, lowercase email) before persistence.
- Hash passwords with a strong adaptive algorithm and a per-user salt.
- Issue a signed token or server session on login; set a sensible expiry.
- Return correct status codes (`201` , `200` , `400` , `401` , `409`) and consistent error shapes.
- Protect against timing attacks by comparing hashes in constant time.

Front-End Requirements

- Registration and login forms with **inline, real-time validation** (email format, password rules).
- Disable submit while a request is in flight; show a loading indicator.
- On success, persist the token/session securely and redirect to a protected landing page.
- On failure, display server-side error messages gracefully (e.g., "Email already in use").
- Provide a link to toggle between the register and login views.

Non-Functional / Security

- Never log raw passwords or tokens.
- Enforce HTTPS assumption; do not expose hashes in any API response.
- Rate-limit or lock out repeated failed logins (design note is acceptable).

Edge Cases to Handle

- Email with different casing/whitespace treated as the same account.
- Empty or missing fields.
- Extremely long input values.
- Submitting the login form before registration exists.

Unit Test Scenarios (Given / When / Then)

#	Given	When	Then
1	A valid new email + password	Register is called	User is created and hash ≠ plaintext
2	An already-registered email	Register is called	Response is `409`, no duplicate created
3	An invalid email or short password	Register is called	Response is `400` with field errors
4	A registered user + correct password	Login is called	A token/session with expiry is returned
5	A registered user + wrong password	Login is called	Response is `401` with a generic message
6	A stored hash + correct password	Verify is called	Verification returns true (round-trip)

Deliverables

- Back-end endpoints with validation, hashing, and token/session issuance.
- Front-end register + login flow wired to the API.
- Unit tests covering all scenarios above (happy + negative paths).
- Short README: how to run the app and the tests, plus your completed estimation table.